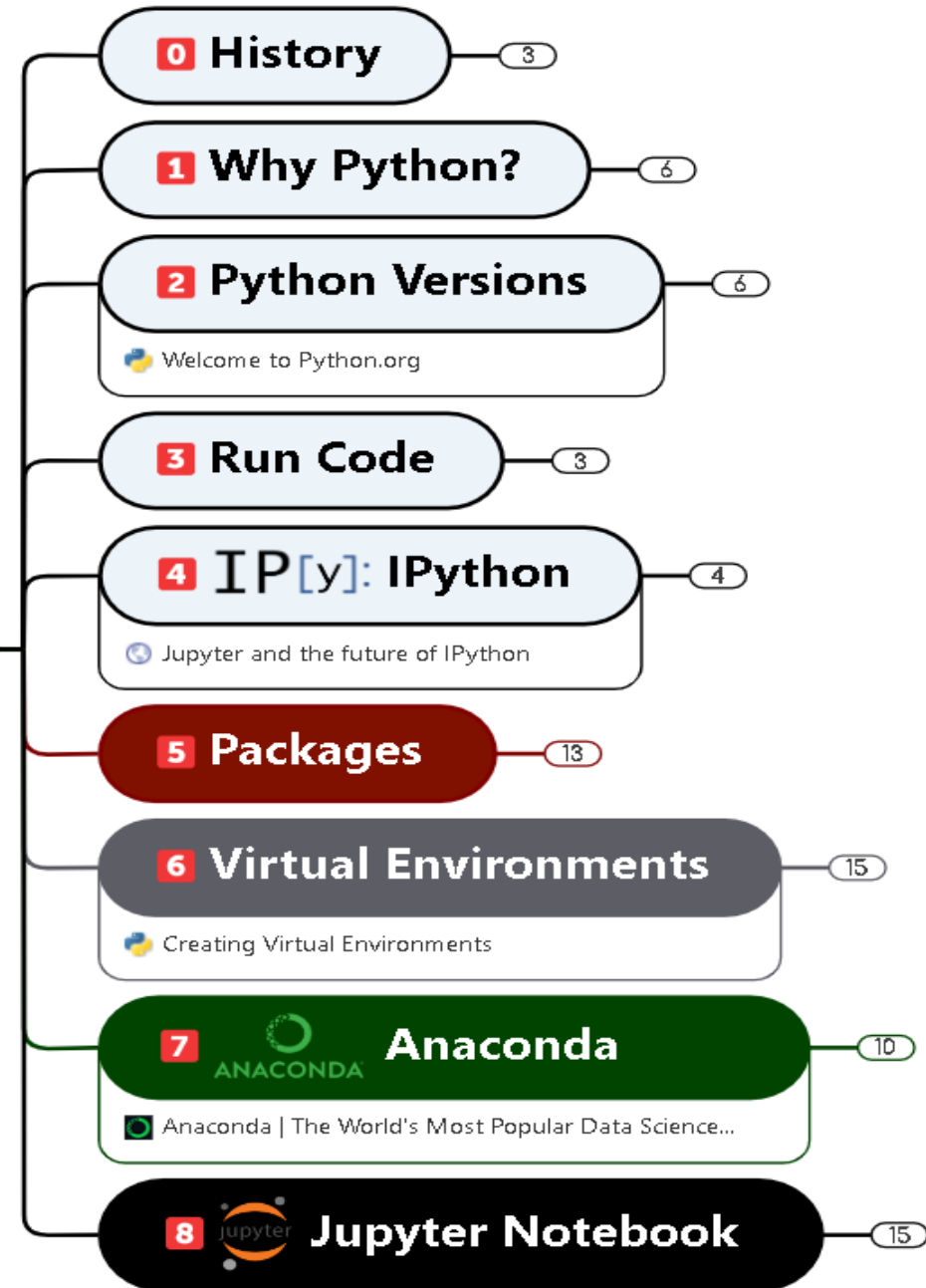


Intro to Python !!

By :Fares Ahmed

Contents

No	Topic taught
1	Variables and Data types
2	Sequences (Strings , lists , tuples , dictionary and sets)
3	..
4	...
5	
n	





0 History



Guido Van Rossum, 1991

ABC Language

Written in C

1 Why Python?

General Purpose

High Level

Easy

Dynamically Typed

Community Support

Huge Library

2 Python Versions

 [Welcome to Python.org](https://www.python.org)

3.10.1

3

2.7.8

...



3 Run Code

Interactive Mode >>>

Script Mode

VSCode

4 IP[y]: IPython

Jupyter and the future of IPython

```
$ sudo apt install ipython3
```

```
import matplotlib.pyplot as plt  
plt.plot([1, 2, 3, 4])  
plt.ylabel('some numbers')  
plt.show()
```

Auto complete

magic functions

5 Packages

What is a Python Package?

The Python Standard Library

 The Python Standard Library

Where are the packages?

The Python Package Index

 PyPI · The Python Package Index

Import External Packages



6 Virtual Environments

Creating Virtual Environments

What are Virtual Environments?

Why and When use Virtual Environments?

Create Virtual Environment

Activate Virtual Environment

Deactivate Virtual Environment

What does Activation mean?

Test different venv and different packages

Test in VSCode

What is Anaconda?

Install Anaconda

Installing on Linux

Explore Anaconda Navigator

conda

pip

Cheatsheet

venv

Important Visualization Libraries

All Tools — PyViz

Launch from Anaconda Navigator

Launch from Anaconda prompt

Explore the Interface

Cell types

nbextensions

7 Anaconda

Anaconda | The World's Most Popular Data Science...

8 Jupyter Notebook



Python Installation

Python >>> Downloads >>> Windows

Python Releases for Windows

- [Latest Python 3 Release - Python 3.7.2](#)
- [Latest Python 2 Release - Python 2.7.15](#)
- [Python 3.8.0a1 - 2019-02-03](#)
 - [Download Windows x86 web-based installer](#)
 - [Download Windows x86 executable installer](#)
 - [Download Windows x86 embeddable zip file](#)
 - [Download Windows x86-64 web-based installer](#)
 - [Download Windows x86-64 executable installer](#)
 - [Download Windows x86-64 embeddable zip file](#)

Python 3.7.2 (64-bit) Setup



python
for
windows

Install Python 3.7.2 (64-bit)

Select Install Now to install Python with default settings, or choose Customize to enable or disable features.



Install Now

C:\Users\anirudh_c_rao\AppData\Local\Programs\Python\Python37

Includes IDLE, pip and documentation

Creates shortcuts and file associations



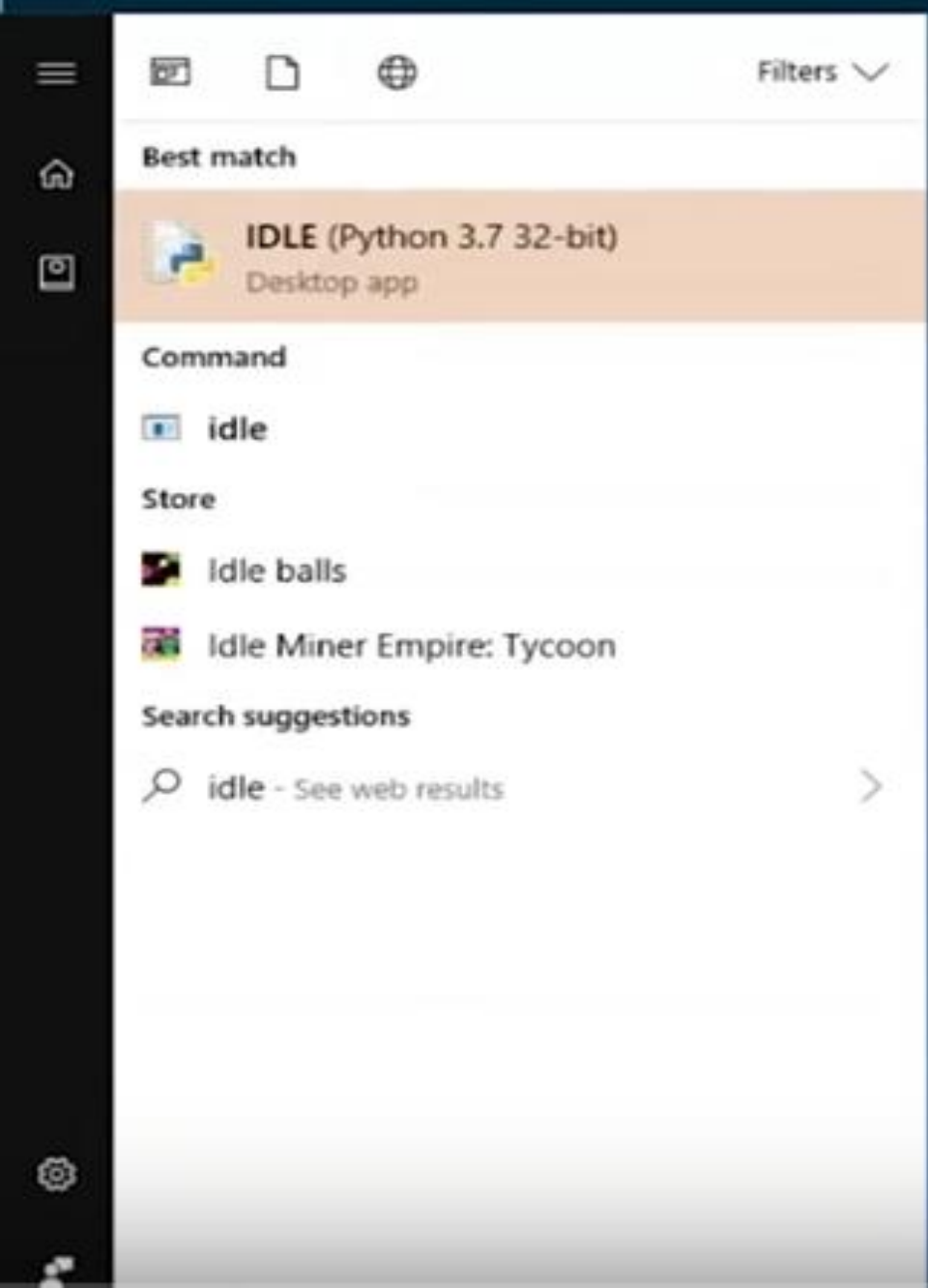
Customize installation

Choose location and features

☒ Install launcher for all users (recommended)

☒ Add Python 3.7 to PATH

Cancel



Installing Python

Python 3.7.0 Shell

File Edit Shell Debug Options Window Help

Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32

Type "copyright", "credits" or "license()" for more information.

```
>>> print("Hello Edureka!")
```

```
Hello Edureka!
```

```
>>>
```

I

Variables and Data types

Variables

```
13
```

```
13
```

```
29 + 85
```

```
114
```

```
12.8
```

```
12.8
```

```
5 * (12.7 - 4) / 2
```

```
21.75
```

```
'Hello World'
```

```
'Hello World'
```



Run



Code



In [1]: x

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-1-6fcf9dfbd479> in <module>()  
----> 1 x  
  
NameError: name 'x' is not defined
```

In [2]: x=10

In [3]: y=20

In [4]: z=x+y

In [5]: print(z)

30

In [6]: Y

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-6-c881daf1af41> in <module>()  
----> 1 Y  
  
NameError: name 'Y' is not defined
```

```
In [7]: type(x)
```

```
Out[7]: int
```

```
In [8]: y=12.5  
type(y)
```

```
Out[8]: float
```

```
In [9]: z=10j  
type(z)
```

```
Out[9]: complex
```

```
In [11]: num=10>9  
type(num)
```

```
Out[11]: bool
```

```
In [12]: num
```

```
Out[12]: True
```

Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

```
x = 10

x += 10 # x = x + 10
print(x)

x -= 10 # x = x - 10
print(x)

x *= 10 # x = x * 10
print(x)

x **= 10
print(x)
```


Arithmetic Operators

Operation	Result
<code>x + y</code>	sum of <code>x</code> and <code>y</code>
<code>x - y</code>	difference of <code>x</code> and <code>y</code>
<code>x * y</code>	product of <code>x</code> and <code>y</code>
<code>x / y</code>	quotient of <code>x</code> and <code>y</code>
<code>x // y</code>	floored quotient of <code>x</code> and <code>y</code>
<code>x % y</code>	remainder of <code>x / y</code>
<code>-x</code>	<code>x</code> negated
<code>+x</code>	<code>x</code> unchanged
<code>abs(x)</code>	absolute value or magnitude of <code>x</code>
<code>int(x)</code>	<code>x</code> converted to integer
<code>float(x)</code>	<code>x</code> converted to floating point
<code>complex(re, im)</code>	a complex number with real part <i>re</i> , imaginary part <i>im</i> . <i>im</i> defaults to zero.
<code>c.conjugate()</code>	conjugate of the complex number <code>c</code>
<code>divmod(x, y)</code>	the pair <code>(x // y, x % y)</code>
<code>pow(x, y)</code>	<code>x</code> to the power <code>y</code>
<code>x ** y</code>	<code>x</code> to the power <code>y</code>

Multiplication (`*`)

```
7 * 4
```

28

Exponentiation (`**`)

```
2 ** 10
```

1024

```
9 ** (1 / 2)
```

3.0

True Division (`/`) vs. Floor Division (`//`)

```
7 / 4
```

1.75

```
7 // 4 # 1 3/4 -- 1 - 1.75 - 2
```

1

```
3 // 5
```

0

Getting Input from the User

Using `input()` built-in function

```
name = input("What's your name? ")
```

What's your name?

name

..

```
print(name)
```

```
name = input("What's your name? ")  
print('Hello Mr. ' + name)
```

What's your name?
Hello Mr.

Function `input()` Always Returns a String

```
value1 = input('Enter first number: ')\nvalue2 = input('Enter second number: ')\n\nvalue1 + value2
```

```
Enter first number: 12\nEnter second number: 43
```

```
'1243'
```

```
name = input('What is your name? ')\nage = input('And what is your age?')\n\nprint ('Hi',name,',you are ',age, 'years old')
```

```
What is your name? oooo\nAnd what is your age?iiiii\nHi oooo ,you are  iiii years old
```

```
value = input('Enter an integer: ')
```

Enter an integer: 88

```
value
```

```
'88'
```

```
value = int(value)
```

Casting

```
value
```

```
88
```

```
another_value = int(input('Enter another integer: '))
```

Enter another integer: 990

```
another_value
```

```
990
```

Dynamic Typing

```
type(x) ← x = 5
```

```
int
```

```
x = 4.1
```

```
type(x)
```

```
float
```

```
x = 'dog'
```

```
type(x)
```

```
str
```

```
x: int = 79
```

```
x
```

```
79
```

```
type(x)
```

```
int
```

String and Numbers as Objects

Strings as objects

```
: name = "ada lovelace"  
print(name.title()) # title() is called a method
```

Ada Lovelace

```
: name = "Ada Lovelace"  
print(name.upper())  
print(name.lower())
```

ADA LOVELACE
ada lovelace

```
: name = "ada lovelace"  
print(name.title())  
first_name = "ada"  
last_name = "lovelace"  
full_name = first_name + " " + last_name  
print(full_name)
```

Ada Lovelace
ada lovelace

Numbers as objects

```
x = 109
```

```
x.bit_length()
```

```
7
```

```
bin(x)
```

```
'0b1101101'
```

```
float(x) # int()
```

```
109.0
```

```
z = 1.7e3
```

```
print(z)
```

```
int(z).bit_length()
```

```
1700.0
```

```
11
```

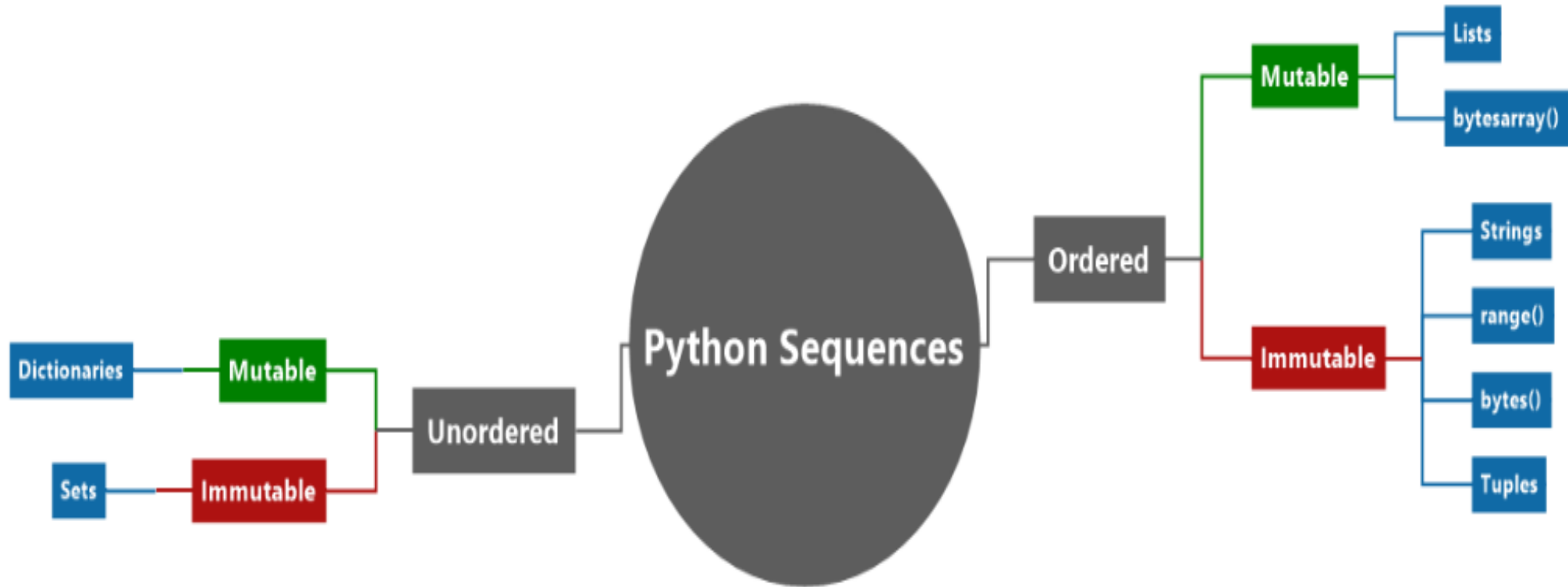
```
bin(int(z))
```

```
'0b11010100100'
```


Sequences

[Sequences]

Containers that hold some values



Python *Ordered* Sequences

- Strings
- Lists
- Tuples
- range()
- Bytes Sequences
- Bytes Array

Python *Unordered* Sequences

- Sets
- Dictionaries

List

ordered sequence

What Is a List?

Equivalent but not equal to arrays

Python doesn't have arrays as part of the standard types, but has array module in the Standard Library

[Python array module](#)

```
In [256]: numbers = [54, -12, 0, 77, 1976]
          numbers
```

```
Out[256]: [54, -12, 0, 77, 1976]
```

```
In [257]: print(type(numbers))

<class 'list'>
```

```
In [258]: misc = [(1,2), 'Ahmed', 47, [25, -9]]
          misc
```

```
Out[258]: [(1, 2), 'Ahmed', 47, [25, -9]]
```

```
In [259]: print(type(misc))

<class 'list'>
```

```
In [260]: bicycles = ['trek', 'cannondale', 'redline', 'specialized']
          print(bicycles)

['trek', 'cannondale', 'redline', 'specialized']
```

```
In [261]: # Implicit line joining
bicycles = ['trek',
            'cannondale',
            'redline',
            'specialized']
print(bicycles)

['trek', 'cannondale', 'redline', 'specialized']
```

Accessing Elements in a List

```
In [262]: bicycles = ['trek', 'cannondale', 'redline', 'specialized']
print(bicycles[1])

cannondale
```

```
In [263]: type(bicycles[1])
```

```
Out[263]: str
```

```
In [264]: bicycles = ['trek', 'cannondale', 'redline', 'specialized']
print(bicycles[1].title())
```

Cannondale

Index Positions Start at 0, Not 1

```
55]: bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[1])  
print(bicycles[2])
```

```
cannondale  
redline
```

```
56]: print(bicycles[5])
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-266-aa8b269c3522> in <module>  
----> 1 print(bicycles[5])  
  
IndexError: list index out of range
```

```
57]: bicycles = ['trek', 'cannondale', 'redline', 'specialized']  
print(bicycles[-1])  
print(bicycles[-2])
```

```
specialized  
redline
```

Changing, Adding, and Removing Elements

Modifying Elements in a List

```
[269]: # This is what Mutable means
motorcycles = ['honda', 'yamaha', 'suzuki']
print(motorcycles)

['honda', 'yamaha', 'suzuki']
```

```
[270]: motorcycles[0] = 'ducati'
print(motorcycles)

['ducati', 'yamaha', 'suzuki']
```

Adding Elements to the End of a List

```
[271]: motorcycles = ['honda', 'yamaha', 'suzuki']
print(motorcycles)

motorcycles.append('ducati')
print(motorcycles)

['honda', 'yamaha', 'suzuki']
['honda', 'yamaha', 'suzuki', 'ducati']
```



```
[272]: motorcycles = []

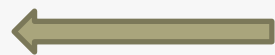
motorcycles.append('honda')
motorcycles.append('yamaha')
motorcycles.append('suzuki')

print(motorcycles)

['honda', 'yamaha', 'suzuki']
```

```
[273]: # Concatinating Lists
```

```
num1 = [12, 423, 109]
num2 = [99, -24]
num = num1 + num2
num
```



Concatenating Lists

```
[273]: [12, 423, 109, 99, -24]
```

```
[274]: num = [2,4,7]
num * 2
```

```
[274]: [2, 4, 7, 2, 4, 7]
```

We can use the method `extend` to add new elements to the list:

```
In [6]: # Use extend to add elements to list
```

```
L = [ "Michael Jackson", 10.2]
L.extend(['pop', 10])
L
```

```
Out[6]: ['Michael Jackson', 10.2, 'pop', 10]
```

Diff between **append** and **extend** when add more than one element to a list

Another similar method is `append`. If we apply `append` instead of `extend`, we add one element to the list:

```
In [7]: # Use append to add elements to list
```

```
L = [ "Michael Jackson", 10.2]
L.append(['pop', 10])
L
```

```
Out[7]: ['Michael Jackson', 10.2, ['pop', 10]]
```

Inserting Elements into a List

```
5]: motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)  
  
motorcycles.insert(2, 'ducati')  
print(motorcycles)
```

```
['honda', 'yamaha', 'suzuki']
```

```
['honda', 'yamaha', 'ducati', 'suzuki']
```

Removing an Item Using the del Statement

```
276]: motorcycles = ['honda', 'yamaha', 'suzuki']  
      print(motorcycles)  
  
      del motorcycles[0]  
      print(motorcycles)
```

```
['honda', 'yamaha', 'suzuki']  
['yamaha', 'suzuki']
```

```
277]: motorcycles = ['honda', 'yamaha', 'suzuki']  
      print(motorcycles)  
  
      del motorcycles[1]  
      print(motorcycles)
```

```
['honda', 'yamaha', 'suzuki']  
['honda', 'suzuki']
```

Removing an Item Using the pop() Method

```
278]: motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles)
```

```
popped_motorcycle = motorcycles.pop()
```

```
print(motorcycles)  
print(popped_motorcycle)
```

```
['honda', 'yamaha', 'suzuki']  
['honda', 'yamaha']  
suzuki
```

```
279]: motorcycles = ['honda', 'yamaha', 'suzuki']  
last_owned = motorcycles.pop()
```

```
print("The last motorcycle I owned was a " + last_owned.title() + ".")
```

```
The last motorcycle I owned was a Suzuki.
```

Popping Items from any Position in a List

```
[280]: motorcycles = ['honda', 'yamaha', 'suzuki']  
first_owned = motorcycles.pop(0)  
  
print('The first motorcycle I owned was a ' + first_owned.title() + '.')  
  
The first motorcycle I owned was a Honda.
```

Removing an Item by Value

```
[281]: motorcycles = ['honda', 'yamaha', 'suzuki', 'ducati']  
print(motorcycles)  
  
motorcycles.remove('ducati')  
print(motorcycles)  
  
['honda', 'yamaha', 'suzuki', 'ducati']  
['honda', 'yamaha', 'suzuki']
```

Organizing a List

Sorting a List Permanently with the sort() Method

```
] : cars = ['bmw', 'audi', 'toyota', 'subaru']  
cars.sort()  
print(cars)
```

```
['audi', 'bmw', 'subaru', 'toyota']
```

```
] : cars = ['bmw', 'audi', 'toyota', 'subaru']  
cars.sort(reverse=True)  
print(cars)
```

```
['toyota', 'subaru', 'bmw', 'audi']
```

Sorting a List Temporarily with the sorted() Function

```
[85]: cars = ['bmw', 'audi', 'toyota', 'subaru']  
print("Here is the original list:")  
print(cars)  
  
print("\nHere is the sorted list:")  
print(sorted(cars))  
  
print("\nHere is the original list again:")  
print(cars)
```

Here is the original list:
['bmw', 'audi', 'toyota', 'subaru']

Here is the sorted list:
['audi', 'bmw', 'subaru', 'toyota']

Here is the original list again:
['bmw', 'audi', 'toyota', 'subaru']

Printing a List in Reverse Order

```
In [286]: cars = ['bmw', 'audi', 'toyota', 'subaru']  
print(cars)  
  
cars.reverse()  
print(cars)  
  
['bmw', 'audi', 'toyota', 'subaru']  
['subaru', 'toyota', 'audi', 'bmw']
```

Finding the Length of a List

```
In [287]: cars = ['bmw', 'audi', 'toyota', 'subaru']  
len(cars)
```

```
Out[287]: 4
```

`list()` constructor function

```
In [288]: list([1,3,4,5])
```

```
Out[288]: [1, 3, 4, 5]
```

```
In [289]: type(list([1,3,4,5]))
```

```
Out[289]: list
```

```
In [290]: list('Python')
```

```
Out[290]: ['P', 'y', 't', 'h', 'o', 'n']
```

list() with enumerate()

```
[291]: values = ['first', 'second', 'third', 'fourth']  
for index in range(len(values):  
    value = values[index]  
    print(index, value)
```

```
0 first  
1 second  
2 third  
3 fourth
```

```
[292]: enumerate(values)
```

```
Out[292]: <enumerate at 0x1713d1ff780>
```

```
[293]: list(enumerate(values)) # unpacking sequences
```

```
Out[293]: [(0, 'first'), (1, 'second'), (2, 'third'), (3, 'fourth')]
```

```
[294]: values = ['first', 'second', 'third', 'fourth']  
for index, value in enumerate(values):  
    print(index, value)
```

```
0 first  
1 second  
2 third  
3 fourth
```

Avoiding Index Errors When Working with Lists

```
5]: #This will generate 'index out of range error'  
motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles[3])
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-295-28bbc56f7233> in <module>  
      1 #This will generate 'index out of range error'  
      2 motorcycles = ['honda', 'yamaha', 'suzuki']  
----> 3 print(motorcycles[3])
```

IndexError: list index out of range

```
6]: motorcycles = ['honda', 'yamaha', 'suzuki']  
print(motorcycles[-1])
```

suzuki

```
7]: #This will generate 'index out of range error'  
motorcycles = []  
  
len(motorcycles) —————> Print 0  
print(motorcycles[-1])
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-297-b5d9fcb03161> in <module>  
      4 len(motorcycles)  
      5  
----> 6 print(motorcycles[-1])
```

IndexError: list index out of range

Making Numerical Lists

Using the `range()` Function

```
3]: list(range(5))
```

```
3]: [0, 1, 2, 3, 4]
```

```
9]: print(type(range(5)))
```

```
<class 'range'>
```

Slicing (start , stop , step)

```
9]: # slicing (start, stop, step)  
list(range(20, 2, -2))
```

```
9]: [20, 18, 16, 14, 12, 10, 8, 6, 4]
```

```
1]: for value in range(1,5):  
    print(value)
```

```
1  
2  
3  
4
```

Using `range()` to Make a List of Numbers

```
numbers = list(range(1,6))  
print(numbers)
```

[1, 2, 3, 4, 5]

```
even_numbers = list(range(2,11,2))  
print(even_numbers)
```

[2, 4, 6, 8, 10]

Squares

```
squares = []  
for value in range(1,11):  
    square = value ** 2  
    squares.append(square)  
print(squares)
```

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Simple Statistics with a List of Numbers

```
digits = [1, 2, 3, 4, 5, 6, 7, 8, 9, 0]

print(min(digits))

print(max(digits))

print(sum(digits))
```

```
0
9
45
```

List Comprehensions

```
squares = []

for sq in range(1,11):
    squares.append(sq**2)

squares
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
squares = [sq**2 for sq in range(1,11)]
print(squares)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Working with part of a List

Slicing a List

```
: players = ['charles', 'martina', 'michael', 'florence', 'eli']  
print(type(players[0:3]))  
print(type(players[0]))  
  
# list[start:stop:step]
```

```
<class 'list'>  
<class 'str'>
```

```
: players = ['charles', 'martina', 'michael', 'florence', 'eli', 'hassan', 'naima']  
print(players[0:5:2])  
  
['charles', 'michael', 'eli']
```

```
: players = ['charles', 'martina', 'michael', 'florence', 'eli']  
print(players[0:4])  
  
['charles', 'martina', 'michael', 'florence']
```

```
: players = ['charles', 'martina', 'michael', 'florence', 'eli']  
print(players[2:])  
  
['michael', 'florence', 'eli']
```


Copying a List

```
] my_foods = ['pizza', 'falafel', 'carrot cake']
friend_foods = my_foods[:]

print("My favorite foods are:")
print(my_foods)

print("\nMy friend's favorite foods are:")
print(friend_foods)
```

Copy by Value

My favorite foods are:
['pizza', 'falafel', 'carrot cake']

My friend's favorite foods are:
['pizza', 'falafel', 'carrot cake']

```
] my_foods = ['pizza', 'falafel', 'carrot cake']

# This doesn't work:
friend_foods = my_foods

friend_foods.append('ice cream')

print("My favorite foods are:")
print(my_foods)

print("\nMy friend's favorite foods are:")
print(friend_foods)
```

Copy by Reference

My favorite foods are:
['pizza', 'falafel', 'carrot cake', 'ice cream']

My friend's favorite foods are:
['pizza', 'falafel', 'carrot cake', 'ice cream']

```
|: my_drinks = ['shai', 'bebsy', '3er2soos']  
  
# this should work:  
friend_drinks = my_drinks.copy()  
  
my_drinks.append('sa7lab')  
  
print("My favorite drinks are:")  
print(my_drinks)  
  
print("\nMy friend's favorite drinks are:")  
print(friend_drinks)
```

My favorite drinks are:
['shai', 'bebsy', '3er2soos', 'sa7lab']

My friend's favorite drinks are:
['shai', 'bebsy', '3er2soos']

Tuple

ordered sequence

Defining a Tuple

```
# Immutable List
dimensions = (200, 50, 100)

print(dimensions[0])
print(dimensions[1])
print(dimensions[2])
```

```
200
50
100
```

Tuple is immutable List

```
# Error
dimensions = (200, 50)
dimensions[0] = 250
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-319-d34bbc27e12d> in <module>
      1 # Error
      2 dimensions = (200, 50)
----> 3 dimensions[0] = 250
```

TypeError: 'tuple' object does not support item assignment

```
two_d = ((0,19), (23,-4), (91,54.2))
two_d
```

```
((0, 19), (23, -4), (91, 54.2))
```

Looping Through All Values in a Tuple

```
dimensions = (200, 50, 92)
for dimension in dimensions:
    print(dimension)
```

200
50
92

```
dimensions.count(200)
```

1

```
dimensions.index(92)
```

2

```
In [8]: # Create your first tuple

tuple1 = ("sara",10,1.2 )
print(tuple1[0])
print (tuple1[-1])
#Concatenate Tuples
tuple2 = tuple1 + ("hard rock", 10)
print(tuple2)
#Slicing
print(tuple2[0:3])
print(len(tuple2))
```

sara

1.2

('sara', 10, 1.2, 'hard rock', 10)

('sara', 10, 1.2)

5

```
In [11]: # Create a nest tuple

NestedT =(1, 2, ("pop", "rock") , (3,4), ("disco", (1,2)))
```

Each element in the tuple including other tuples can be obtained via an index as shown in the figure:

NT =(1, 2, ("pop", "rock") ,(3,4),("disco", (1,2)))



```
In [13]: # Print element on each index
NestedT =(1, 2, ("pop", "rock") , (3,4), ("disco", (1,2)))
print("Element 3, 1 of Tuple: ", NestedT[3][1])
NestedT[2][1][0]
```

Element 3, 1 of Tuple: 4

Out[13]: 'r'

String

ordered sequence

Strings

Really??

- Yes, strings are ordered sequences :)

```
In [324]: string = 'Hello Everyone'
          string[0]
```

```
Out[324]: 'H'
```

```
In [325]: string[0] = 'M' # Error, strings are immutable sequences
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-325-bdac8d80a660> in <module>
----> 1 string[0] = 'M' # Error, strings are immutable sequences

TypeError: 'str' object does not support item assignment
```

```
In [328]: string.count('e')
```

```
Out[328]: 3
```

```
In [329]: string.index('y')
```

```
Out[329]: 10
```

```
In [331]: list(string)
```

```
Out[331]: ['H', 'e', 'l', 'l', 'o', ' ', 'E', 'v', 'e', 'r', 'y', 'o', 'n', 'e']
```

```
In [332]: string[0:5]
```

Indexing

It is helpful to think of a string as an ordered sequence. Each element in the sequence can be accessed using an index represented by the array of numbers:

Name= "Michael Jackson"

M	i	c	h	a	e	l		J	a	c	k	s	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

The first index can be accessed as follows:

[Tip]: Because indexing starts at 0, it means the first index is on the index 0.

```
In [7]: # Print the first element in the string  
  
print(Name[0])
```

M

Negative Indexing

We can also use negative indexing with strings:

Name= "Michael Jackson"

M	i	c	h	a	e	l		J	a	c	k	s	o	n
-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Negative index can help us to count the element from the end of the string.

The last element is given by the index -1:

```
In [3]: # Print the last element in the string
print (Name [-1])

n
```

The first element can be obtained by index -15:

```
In [ ]: # Print the first element in the string
print (Name [-15])
```

String are immutable

```
In [7]: # Print the first element in the string
```

```
print(Name[0])
```

M

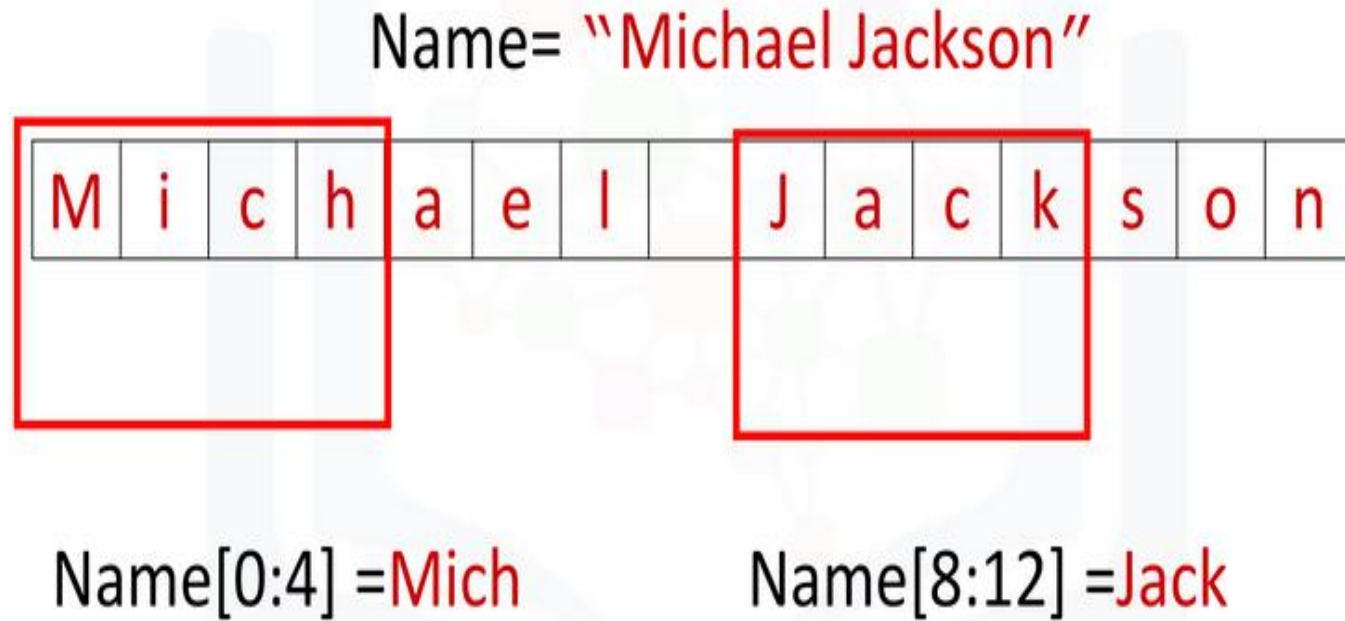
```
In [2]: Name[10]=2
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-2-65a30e5f2a17> in <module>()  
----> 1 Name[10]=2
```

```
TypeError: 'str' object does not support item assignment
```

Slicing

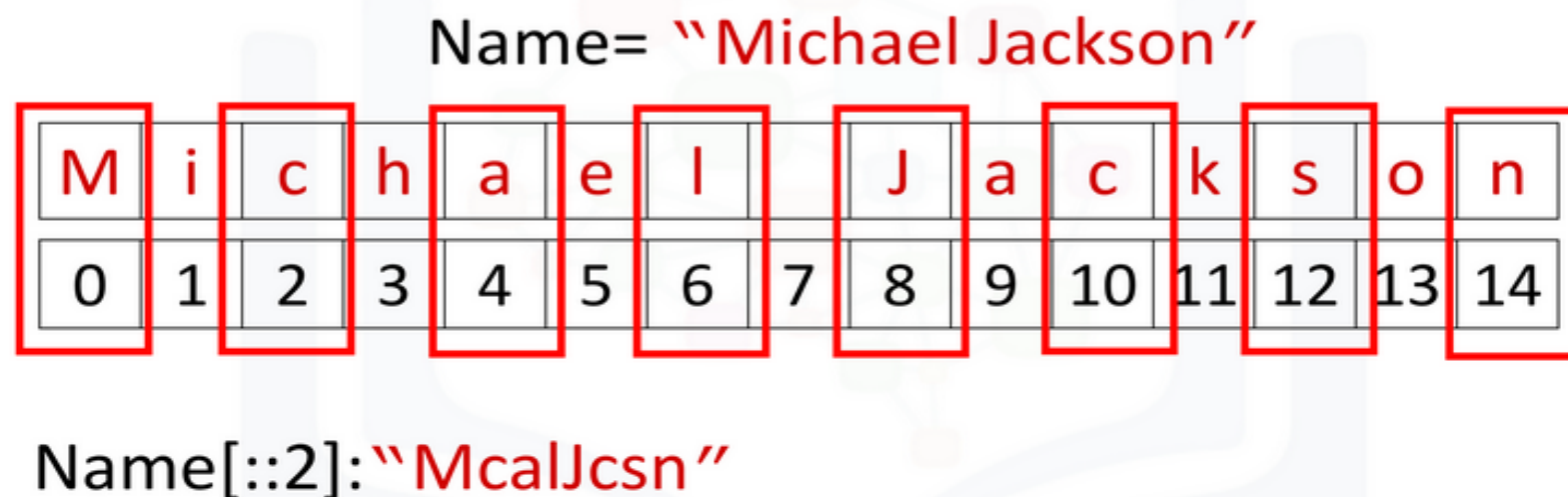
We can obtain multiple characters from a string using slicing, we can obtain the 0 to 4th and 8th to the 12th element:



[Tip]: When taking the slice, the first number means the index (start at 0), and the second number means the length from the index to the last element you want (start at 1)

Stride

We can also input a stride value as follows, with the '2' indicating that we are selecting every second variable:



```
In [5]: # Get every second element. The elements on index 1, 3, 5 ...
```

```
Name[::2]
```

```
Out[5]: 'McalJcsn'
```

We can also incorporate slicing with the stride. In this case, we select the first five elements and then use the stride:

```
In [6]: # Get every second element in the range from index 0 to index 4
```

```
Name[0:5:2]
```

```
Out[6]: 'Mca'
```

Concatenate Strings

We can concatenate or combine strings by using the addition symbols, and the result is a new string that is a combination of both:

```
In [7]: # Concatenate two strings
Statement = Name + "is the best"
Statement
```

```
Out[7]: 'Michael Jacksonis the best'
```

To replicate values of a string we simply multiply the string by the number of times we would like to replicate it. In this case, the number is three. The result is a new string, and this new string consists of three copies of the original string:

```
In [8]: # Print the string for 3 times
3 * "Michael Jackson"
```

```
Out[8]: 'Michael JacksonMichael JacksonMichael Jackson'
```

You can create a new string by setting it to the original variable. Concatenated with a new string, the result is a new string that changes from Michael Jackson to "Michael Jackson is the best".

```
In [9]: # Concatenate strings
Name = "Michael Jackson"
Name = Name + " is the best"
Name
```

```
Out[9]: 'Michael Jackson is the best'
```

String Operations

There are many string operation methods in Python that can be used to manipulate the data. We are going to use some basic string operations on the data.

Let's try with the method `upper`; this method converts lower case characters to upper case characters:

```
In [14]: # Convert all the characters in string to upper case
```

```
A = "Thriller is the sixth studio album"  
print("before upper:", A)  
B = A.upper()  
print("After upper:", B)
```

before upper: Thriller is the sixth studio album

After upper: THRILLER IS THE SIXTH STUDIO ALBUM

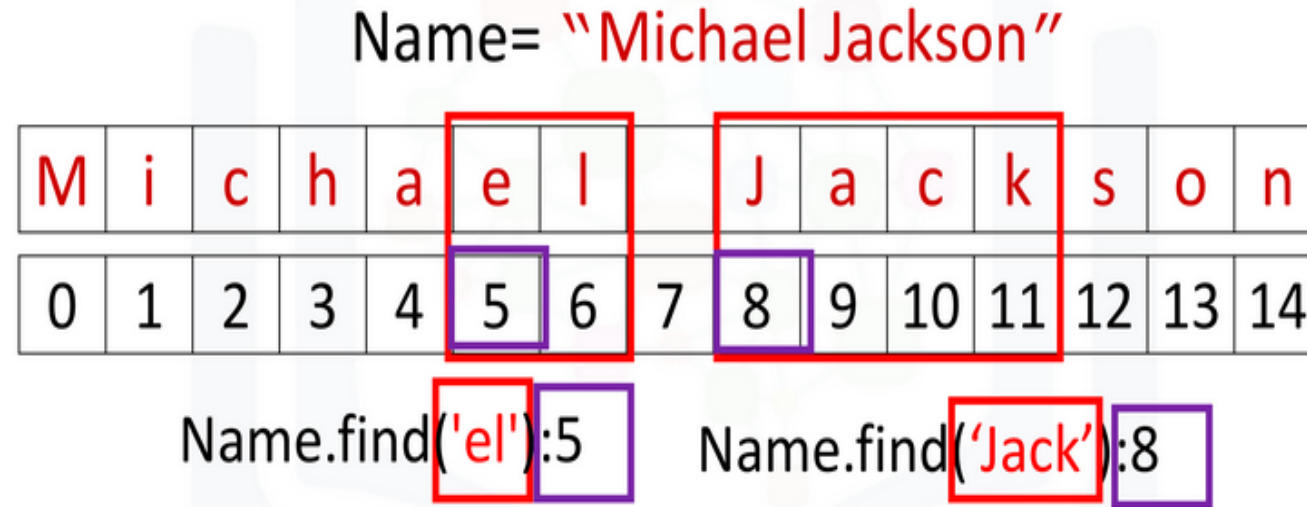
The method `replace` replaces a segment of the string, i.e. a substring with a new string. We input the part of the string we would like to change. The second argument is what we would like to exchange the segment with, and the result is a new string with the segment changed:

```
In [15]: # Replace the old substring with the new target substring is the segment has been found in the string
```

```
A = "Michael Jackson is the best"  
B = A.replace('Michael', 'Janet')  
B
```

```
Out[15]: 'Janet Jackson is the best'
```


The method `find` finds a sub-string. The argument is the substring you would like to find, and the output is the first index of the sequence. We can find the sub-string `jack` or `el`.



```
In [16]: # Find the substring in the string. Only the index of the first element of substring in string will be the output
```

```
Name = "Michael Jackson"  
Name.find('el')
```

```
Out[16]: 5
```

```
In [18]: # Find the substring in the string.
```

```
Name.find('Jasdfasdasdf')
```

```
Out[18]: -1
```

Set

unordered sequence

Sets

```
set1 = {1, 3, 4, 4, 6, 8, 9, 9}  
set1
```

```
{1, 3, 4, 6, 8, 9}
```

```
type(set1)
```

```
set
```

```
my_list = ['one', 'one', 'two', 'three', 'three', 'four']  
my_list
```

```
['one', 'one', 'two', 'three', 'three', 'four']
```

```
# Using the set() function
```

```
my_set = set(my_list)  
my_set
```

```
{'four', 'one', 'three', 'two'}
```

```
list_again = list(set(my_list))  
list_again
```

```
[2, 3, 4]
```

```
dir(set1)
```

```
set1.add(15)  
set1.add(100)  
set1.add(100)  
set1.add(100)  
set1
```

Add Items to set

```
{1, 3, 4, 6, 8, 9, 15, 100}
```

```
# initialize my_set  
my_set = {1, 3}  
print(my_set)
```

```
{1, 3}
```

```
my_set.update([4, 5], {1, 6, 8})  
print(my_set)
```

```
{1, 3, 4, 5, 6, 8}
```

```
my_set.discard(4)  
print(my_set)
```

```
{1, 3, 5, 6, 8}
```

```
: # Set union method
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use | operator UNION
# Output: {1, 2, 3, 4, 5, 6, 7, 8}
print(A | B) # A U B
print(A.union(B))
print(B.union(A))
```

You can use Union and Intersection method between 2 sets and other methods such as difference

```
{1, 2, 3, 4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
```

```
: # Intersection of sets
# initialize A and B
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# use & operator INTERSECTION
# Output: {4, 5}
print(A & B)
print(A.intersection(B))
print(B.intersection(A))
```

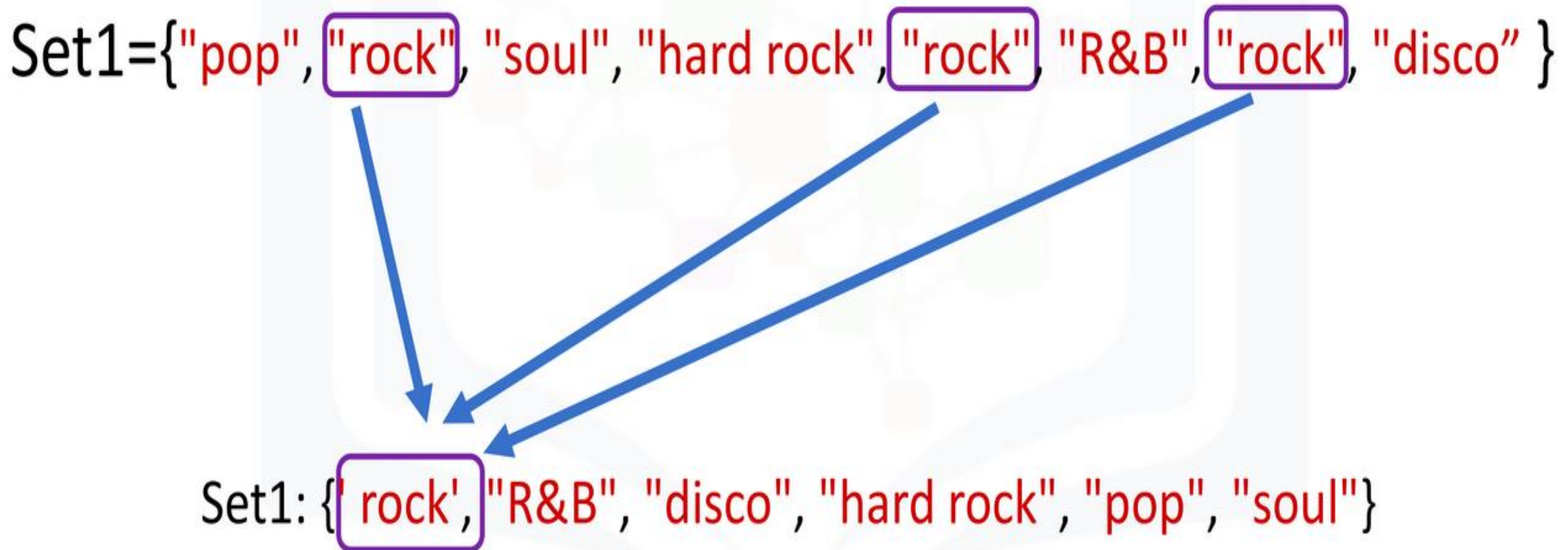
```
{4, 5}
{4, 5}
{4, 5}
```

```
In [1]: # Create a set
```

```
set1 = {"pop", "rock", "soul", "hard rock", "rock", "R&B", "rock", "disco"}  
set1
```

```
Out[1]: {'R&B', 'disco', 'hard rock', 'pop', 'rock', 'soul'}
```

The process of mapping is illustrated in the figure:



Set Operations

Let us go over set operations, as these can be used to change the set. Consider the set **A**:

We can add an element to a set using the `add()` method:

```
In [11]: # Add element to set
myset.add("salsabeel")
myset
```

```
Out[11]: {'abdelrhman', 'salah', 'salsabeel', 'sara'}
```

If we add the same element twice, nothing will happen as there can be no duplicates in a set:

We can remove an item from a set using the `remove` method:

```
In [12]: # Remove the element from set

myset.remove("salah")
myset
```

```
Out[12]: {'abdelrhman', 'salsabeel', 'sara'}
```

We can verify if an element is in the set using the `in` command:

```
In [13]: # Verify if the element is in the set

"sara" in myset
```

```
Out[13]: True
```

Dictionary

unordered sequence

What are Dictionaries?

A dictionary consists of keys and values. It is helpful to compare a dictionary to a list. Instead of the numerical indexes such as a list, dictionaries have keys. These keys are the keys that are used to access values within a dictionary.

List

Index

0	Element 1
1	Element 2
2	Element 3
3	Element 4
.....	

Element

Dictionary

Key: is a index by label

Key 1	Value 1
Key 1	Value 2
Key 2	Value 3
Key 3	Value 4
.....	

Element/Values

Dictionaries

A dictionary is a collection of a key-value pairs.

```
human1 = {'height':180, 'weight':80, 'eye-color':'green', 'language':'mongolian'}
```

```
print(human1)
print(type(human1))
```

```
{'height': 180, 'weight': 80, 'eye-color': 'green', 'language': 'mongolian'}
<class 'dict'>
```

human1 vs print(human1)

```
human1
{'height': 180, 'weight': 80, 'eye-color': 'green', 'language': 'mongolian'}
```

```
print(human1.keys())
print(human1.values())
```

```
dict_keys(['height', 'weight', 'eye-color', 'language'])
dict_values([180, 80, 'green', 'mongolian'])
```

```
len(human1)
```

4

```
human1['sex'] = 'male'
human1
```

```
{'height': 180,
 'weight': 80,
 'eye-color': 'green',
 'language': 'mongolian',
 'sex': 'male'}
```

```
# Modify a value
human1['weight'] = 82
human1
```

```
{'height': 180,
 'weight': 82,
 'eye-color': 'green',
 'language': 'mongolian',
 'sex': 'male'}
```

```
my_set = set(human1.values())
my_set
```

```
{180, 82, 'green', 'male', 'mongolian'}
```

Convert to set

An example of a Dictionary Dict :

```
In [2]: # Create the dictionary

Dict = {"key1": 1, "key2": "2", "key3": [3, 3, 3], "key4": (4, 4, 4), ('key5'): 5, (0, 1): 6}
Dict
```

```
Out[2]: {'key1': 1,
        'key2': '2',
        'key3': [3, 3, 3],
        'key4': (4, 4, 4),
        'key5': 5,
        (0, 1): 6}
```

The keys can be strings:

```
In [3]: # Access to the value by the key

Dict["key1"]
```

```
Out[3]: 1
```

Keys can also be any immutable object such as a tuple:

```
In [5]: # Access to the value by the key

Dict[(0, 1)]
```

```
Out[5]: 6
```

```
: # list values
human3 = {'name': 'ahmed', 'age': 45, 'language': ['arabic', 'english']}
```

```
: human3['language'][1]
```

```
: 'english'
```

```
: human3['language'][0]
```

```
: 'arabic'
```

```
: human4 = {'name': 'ahmed', 'age': 45, 'language': ['arabic', 'english'], 19:19}
```

```
: # nested dictionaries
```

```
humans = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},
          2: {'name': 'Marie', 'age': '22', 'sex': 'Female'},
          3: {'name': 'shawkat', 'age': '62', 'sex': 'Male'}}
}
```

```
humans
```

```
: {1: {'name': 'John', 'age': '27', 'sex': 'Male'},
   2: {'name': 'Marie', 'age': '22', 'sex': 'Female'},
   3: {'name': 'shawkat', 'age': '62', 'sex': 'Male'}}
```

Nested dict

```
: humans[1]['age']
```

```
: '27'
```

```
: print(humans[1]['name'], humans[2]['name'], humans[3]['name'])
```

```
John Marie shawkat
```

```
humans[3]['name'] = 'rabso'  
humans[3]['age'] = '100'  
humans[3]['sex'] = 'powder'  
humans
```

```
{1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
 2: {'name': 'Marie', 'age': '22', 'sex': 'Female'},  
 3: {'name': 'rabso', 'age': '100', 'sex': 'powder'}}
```

```
humans[4] = {'name': 'Gamal', 'age': '47', 'married': 'No'}  
humans
```

```
{1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
 2: {'name': 'Marie', 'age': '22', 'sex': 'Female'},  
 3: {'name': 'rabso', 'age': '100', 'sex': 'powder'},  
 4: {'name': 'Gamal', 'age': '47', 'married': 'No'}}
```

```
del humans[4]['married']  
humans
```

```
{1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
 2: {'name': 'Marie', 'age': '22', 'sex': 'Female'},  
 3: {'name': 'rabso', 'age': '100', 'sex': 'powder'},  
 4: {'name': 'Gamal', 'age': '47'}}
```

```
del humans[2], humans[4]  
humans
```

```
{1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
 3: {'name': 'rabso', 'age': '100', 'sex': 'powder'}}
```

Thank you

To be continued